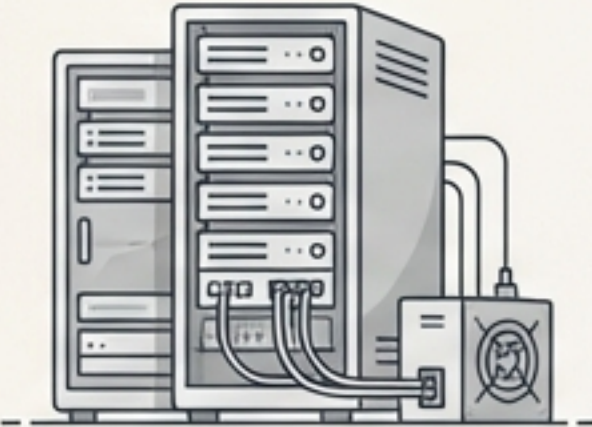


AWS와 Terraform을 통한 자동 인프라 구축: 코드로부터 클라우드까지

현대적인 IT 인프라 관리의 진화와 핵심 전략



패러다임의 전환: 물리적 데이터 센터에서 클라우드

항목 (Criteria)	전통적 데이터 센터 (Traditional Data Center)	클라우드 컴퓨팅 (Cloud Computing)
		
 비용 효율성 (Cost Efficiency)	초기 투자 비용(CapEx)이 높고, 모든 장비와 소프트웨어에 대한 비용을 직접 지불해야 함.	사용한 만큼만 비용을 지불(OpEx)하여 초기 투자 비용을 크게 절감.
 확장성 및 유연성 (Scalability & Flexibility)	확장을 위해 추가 하드웨어 구매가 필요하며, 시간과 비용이 많이 소요됨.	비즈니스 요구 사항에 따라 리소스를 즉시, 쉽게 확장 또는 축소 가능.
 유지 관리 (Maintenance)	모든 하드웨어를 직접 관리해야 하므로 유지 관리 비용과 인력이 많이 필요함.	클라우드 제공업체가 하드웨어를 유지 관리 하므로 IT 팀은 핵심 비즈니스에 집중 가능.

클라우드는 비용, 속도, 민첩성 측면에서 명확한 비즈니스 이점을 제공합니다.

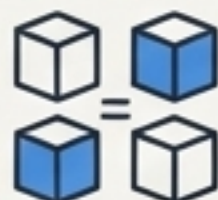
변화의 핵심 동력: 코드형 인프라 (Infrastructure as Code)

laC란 수동 프로세스 대신, 기계가 읽을 수 있는 코드와 스크립트를 사용하여 컴퓨팅 인프라를 구성하고 관리하는 방식입니다.



자동화 (Automation)

인프라 프로비저닝 및 관리 작업을 자동화하여 속도 향상 및 인적 오류 감소.



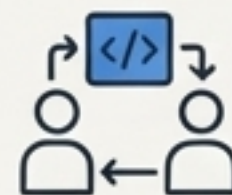
일관성 (Consistency)

코드를 통해 동일한 환경을 반복적으로 배포하여 '내 PC에선 됐는데' 문제를 해결.



버전 관리 (Version Control)

Git과 같은 버전 제어 시스템으로 인프라 변경 내역을 추적하고 감사 가능.



협업 (Collaboration)

팀 내에서 코드를 공유하며 인프라를 함께 구축하고 검토.

“laC는 DevOps 문화 철학을 구현하는 핵심 기술입니다.”

첫 번째 기둥 | 플랫폼: Amazon Web Services (AWS)

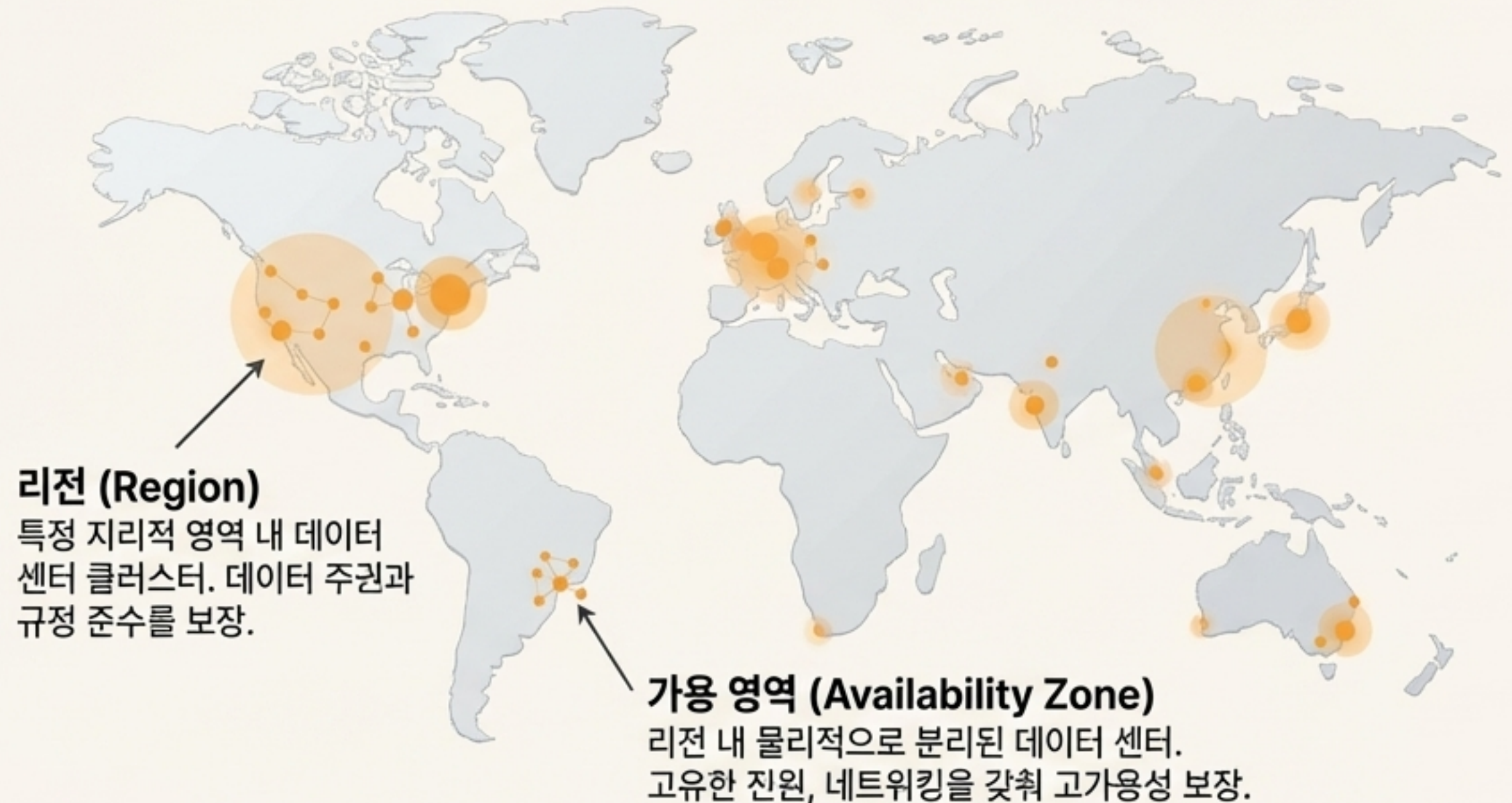
세계 최대의 클라우드 서비스 제공업체이자 클라우드 컴퓨팅 분야의 혁신적 리더.

시장 점유율

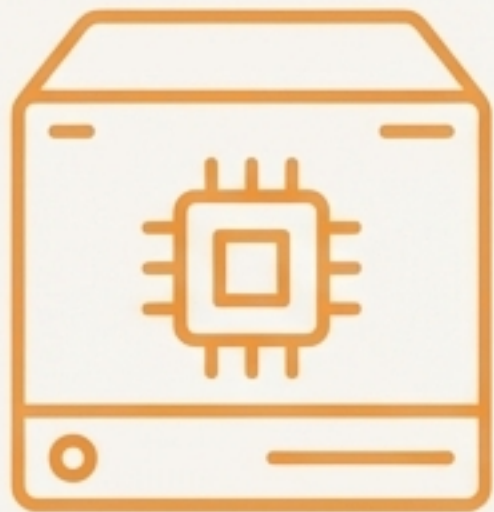
2020년 기준, 공공 클라우드 시장의 41%를 차지하며 압도적인 1위 유지.

성장

2015년 78.8억 달러에서 2020년 453.7억 달러로 매출 급증.



핵심 AWS 빌딩 블록: 인프라의 기본 요소



EC2 (Elastic Compute Cloud)

AWS의 핵심 컴퓨팅 서비스로, 다양한 용도의 가상 서버(인스턴스)를 제공합니다.



S3 (Simple Storage Service)

대규모 데이터를 안전하게 저장하고 액세스할 수 있는 객체 스토리지 서비스입니다.



RDS (Relational Database Service)

MySQL, PostgreSQL 등 다양한 엔진을 지원하는 관리형 관계형 데이터베이스 서비스입니다.



VPC (Virtual Private Cloud)

사용자의 AWS 계정 전용 가상 네트워크로, 논리적으로 격리된 공간에서 리소스를 안전하게 실행합니다.

두 번째 기둥 | 설계도: HashiCorp Terraform

HashiCorp에서 오픈소로 개발한 인프라 관리 도구로, 코드를 통해 안전하고 효율적으로 인프라를 구축, 변경 및 버전 관리합니다.



선언적 구문 (Declarative Syntax)

'무엇을' 만들 것인지만 정의하면,
Terraform이 '어떻게' 만들지 결정합니다.



상태 관리 (State Management)

`terraform.tfstate` 파일을 통해
인프라의 현재 상태를 추적하고, 코드와의
차이점을 계산하여 변경 사항을 적용합니다.



멀티 클라우드 (Multi-Cloud)

AWS뿐만 아니라 Azure, Google Cloud 등
다양한 클라우드 제공업체와 사내 솔루션을
하나의 워크플로우로 관리할 수 있습니다.

Terraform은 특정 클라우드에 종속되지 않고, 예측 가능하며 반복 가능한 방식으로 인프라를 자동화합니다.

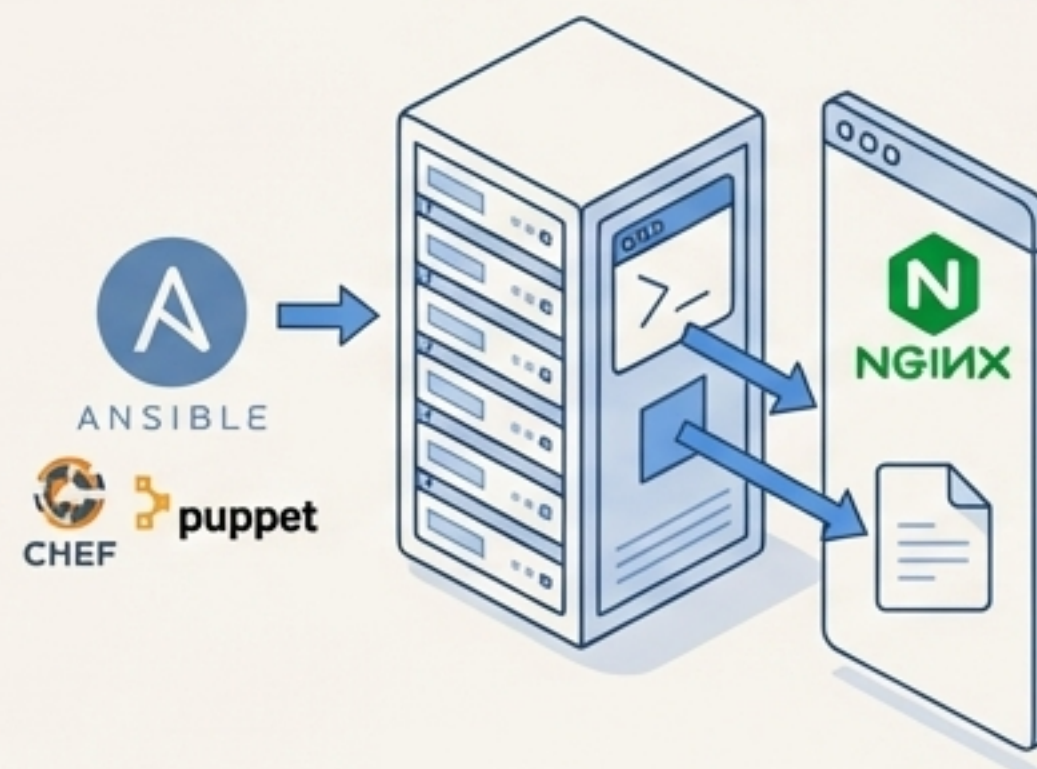
자동화 생태계에서의 Terraform의 역할

프로비저닝 도구 (Provisioning Tool) - Terraform



- **역할:** 인프라 자체를 생성, 수정, 삭제합니다. (서버, 데이터베이스, 네트워크 등)
- **핵심 기능:** 클라우드 API를 사용하여 인프라 리소스를 선언적으로 관리합니다.
- **예시:** "AWS에 2개의 웹 서버 인스턴스와 1개의 로드 밸런서를 생성하라."

구성 관리 도구 (Configuration Management Tool) - Ansible, Chef, Puppet



- **역할 (Role):** 이미 생성된 서버에 소프트웨어를 설치하고 구성합니다.
- **핵심 기능:** 특정 상태에서 머신을 지정된 상태로 유지합니다. (운영용)
- **예시:** "웹 서버에 Nginx를 설치하고 설정 파일을 배포하라."

시너지 (Synergy)

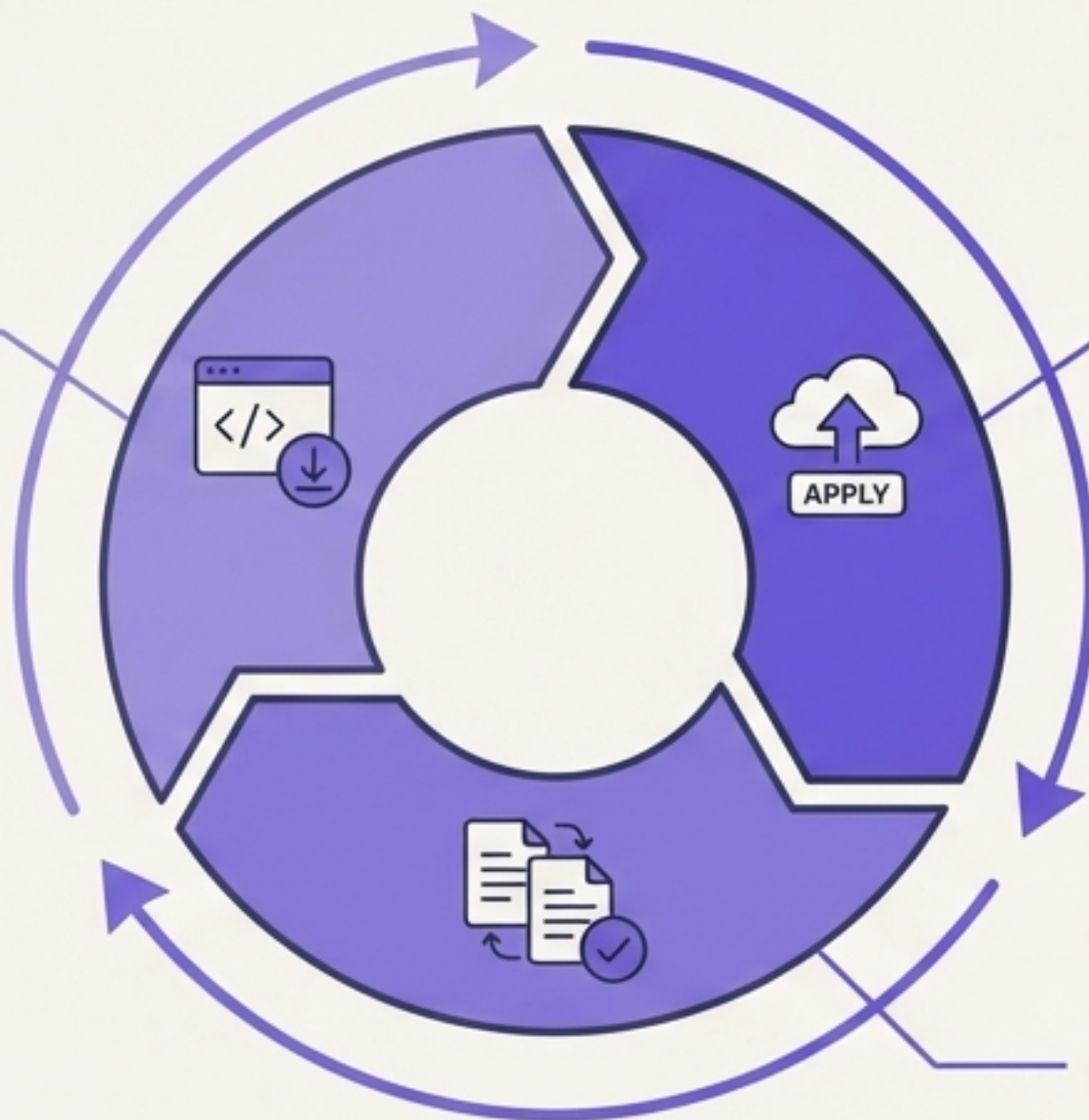
Terraform으로 인프라를 프로비저닝한 후, Ansible과 같은 도구를 사용하여 소프트웨어를 구성하는 것이 일반적인 워크플로우입니다.

Terraform 워크플로우: 안전하고 예측 가능한 3단계 프로세스



작성 (Write / `init`)

HCL(HashiCorp Configuration Language)을 사용하여 인프라 구성(.tf 파일)을 코드로 작성합니다.
`terraform init` 명령어로 작업 디렉터리를 초기화하고 필요한 플러그인을 다운로드합니다.



적용 (Apply / `apply`)

`terraform apply` 명령어로 실행 계획을 승인하면, Terraform이 계획된 변경 사항을 인프라에 정확히 적용합니다.



계획 (Plan / `plan`)

`terraform plan` 명령어를 실행하여 현재 인프라 상태와 코드를 비교합니다. Terraform이 수행할 작업(생성, 수정, 삭제)의 실행 계획을 미리 보여주어 의도치 않은 변경을 방지합니다.

Terraform 구성 파일의 해부학: EC2 인스턴스 생성

```
# 1. 공급자(Provider) 설정
provider "aws" {
  region = "ap-northeast-2"
}
```

사용할 클라우드(AWS)와 리전(Region)을 지정합니다. access_key와 secret_key는 보안을 위해 변수 사용을 권장합니다.

```
# 2. 리소스(Resource) 정의
resource "aws_instance" "example" {
  ami           = "ami-00edfb46b107f643c"
  instance_type = "t2.micro"

  tags = {
    Name = "My First Terraform Instance"
  }
}
```

생성할 인프라 자원을 선언합니다.
형식: `resource "<resource\_type>" "<resource\_name>"`

Amazon Machine Image (AMI) ID를 지정합니다.

인스턴스의 사양(크기)을 지정합니다.

식별을 위한 태그를 추가합니다.

재사용성과 보안 강화: 변수(Variables)와 출력(Outputs)

변수 (Variables) - 왜 필요한가?

민감한 정보(API 키 등)를 코드에서 분리하고, 리전별 AMI처럼 자주바뀌는 값을 쉽게 관리하여 코드의 재사용성을 높입니다.

```
# vars.tf - 변수 선언
variable "aws_region" {
  description = "The AWS region to deploy"
  type        = string
  default     = "ap-northeast-2"
}
```

```
# main.tf - 변수 사용
provider "aws" {
  region = var.aws_region
}
```

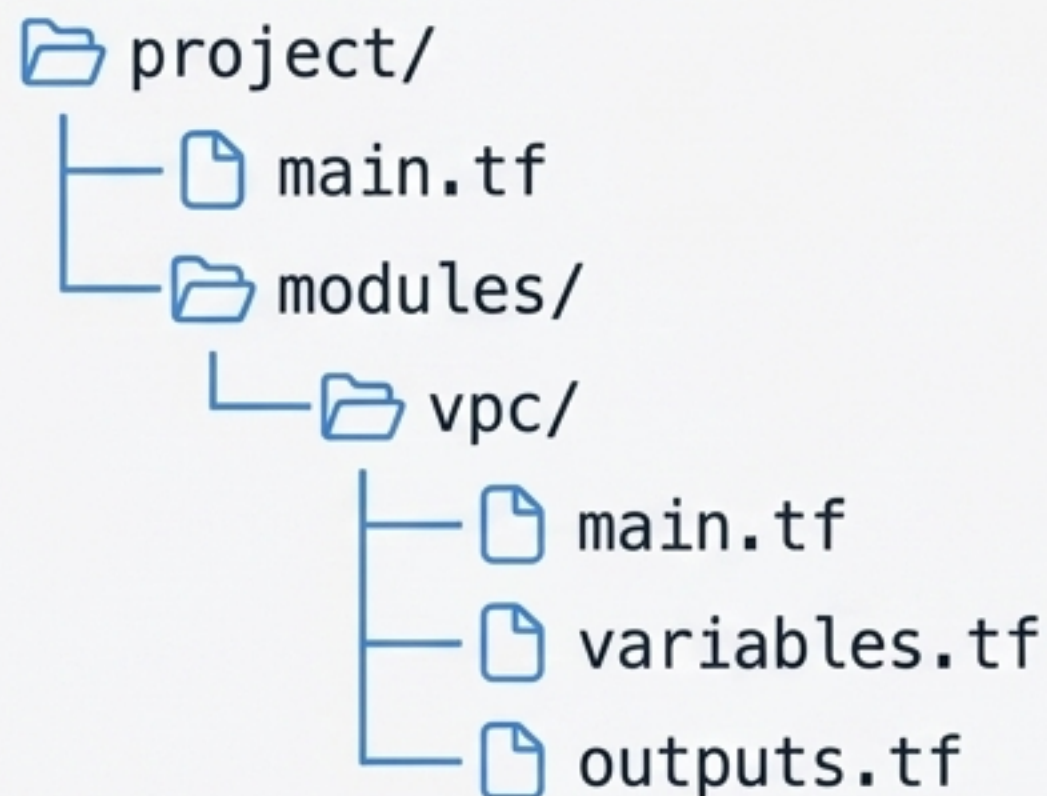
출력 (Outputs) - 왜 필요한가?

Terraform으로 생성된 리소스의 속성(예: EC2 인스턴스의 IP 주소)을 터미널에 표시하거나 다른 Terraform 구성에서 사용할 수 있도록 추출합니다.

```
# outputs.tf - 출력 정의
output "instance_public_ip" {
  description = "The public IP address of the EC2 instance."
  value       = aws_instance.example.public_ip
}
```


복잡성 관리의 표준: 모듈(Modules)

모듈은 재사용 가능한 Terraform 구성의 묶음입니다. 프로그래밍 언어의 '함수'처럼, 관련된 리소스 그룹을 하나의 논리적 단위로 캡슐화하여 코드를 체계적으로 구성하고 재사용할 수 있게 합니다.



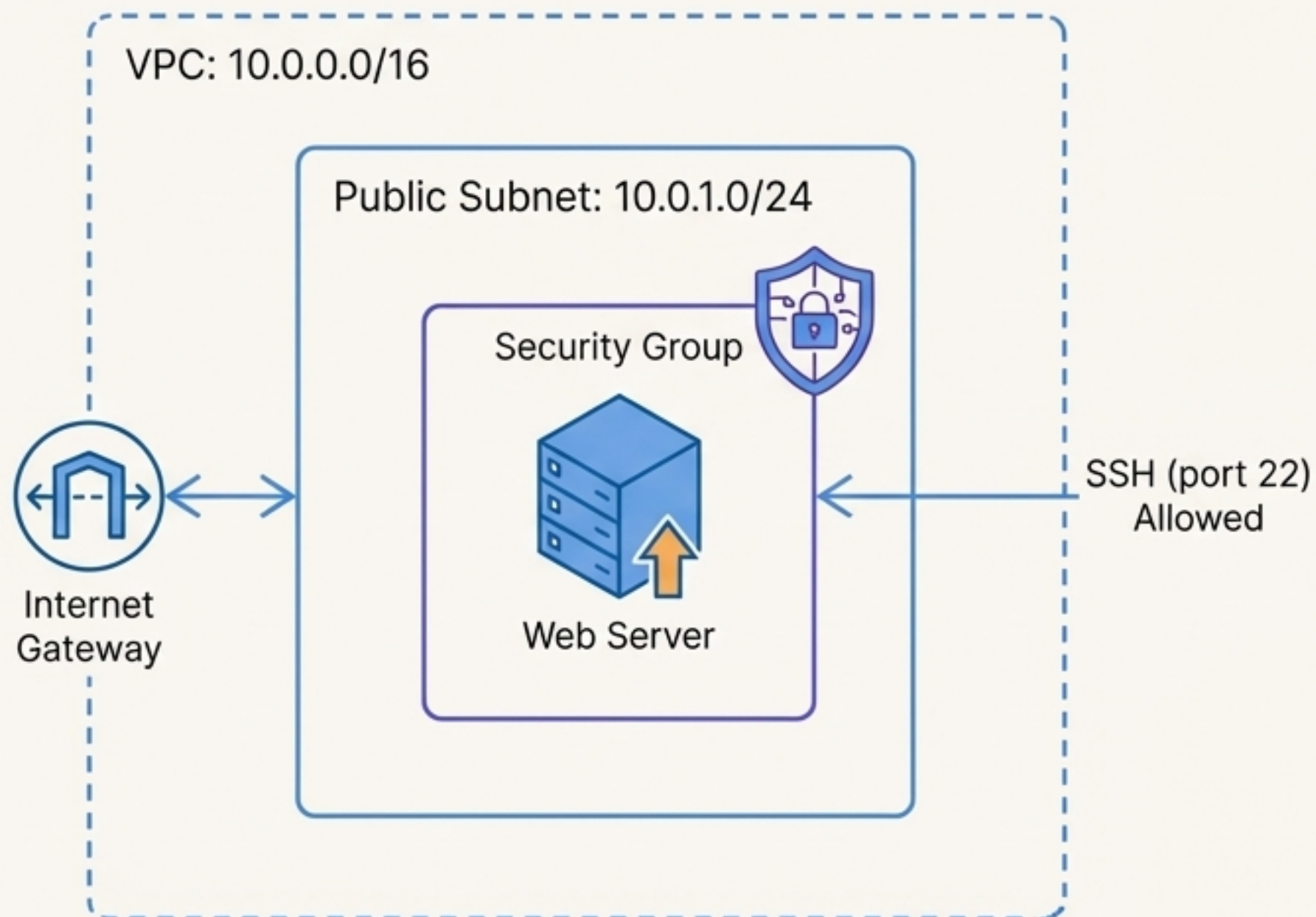
```
# project/main.tf

# 'vpc' 모듈을 호출하여 네트워크를 생성합니다.
# 필요한 값(ip_range)을 인자로 전달합니다.
module "my_vpc" {
    source      = "./modules/vpc"
    ip_range    = "10.0.0.0/16"
}

# 모듈의 출력값을 다른 리소스에서 사용합니다.
resource "aws_instance" "web" {
    subnet_id = module.my_vpc.public_subnet_id
    # ...
}
```

모듈을 통해 인프라 구성 요소를 표준화하고, 팀 간 협업을 촉진하며, 대규모 프로젝트의 유지보수성을 극대화합니다.

아키텍처 청사진: 코드로 VPC와 EC2 인스턴스 구축하기



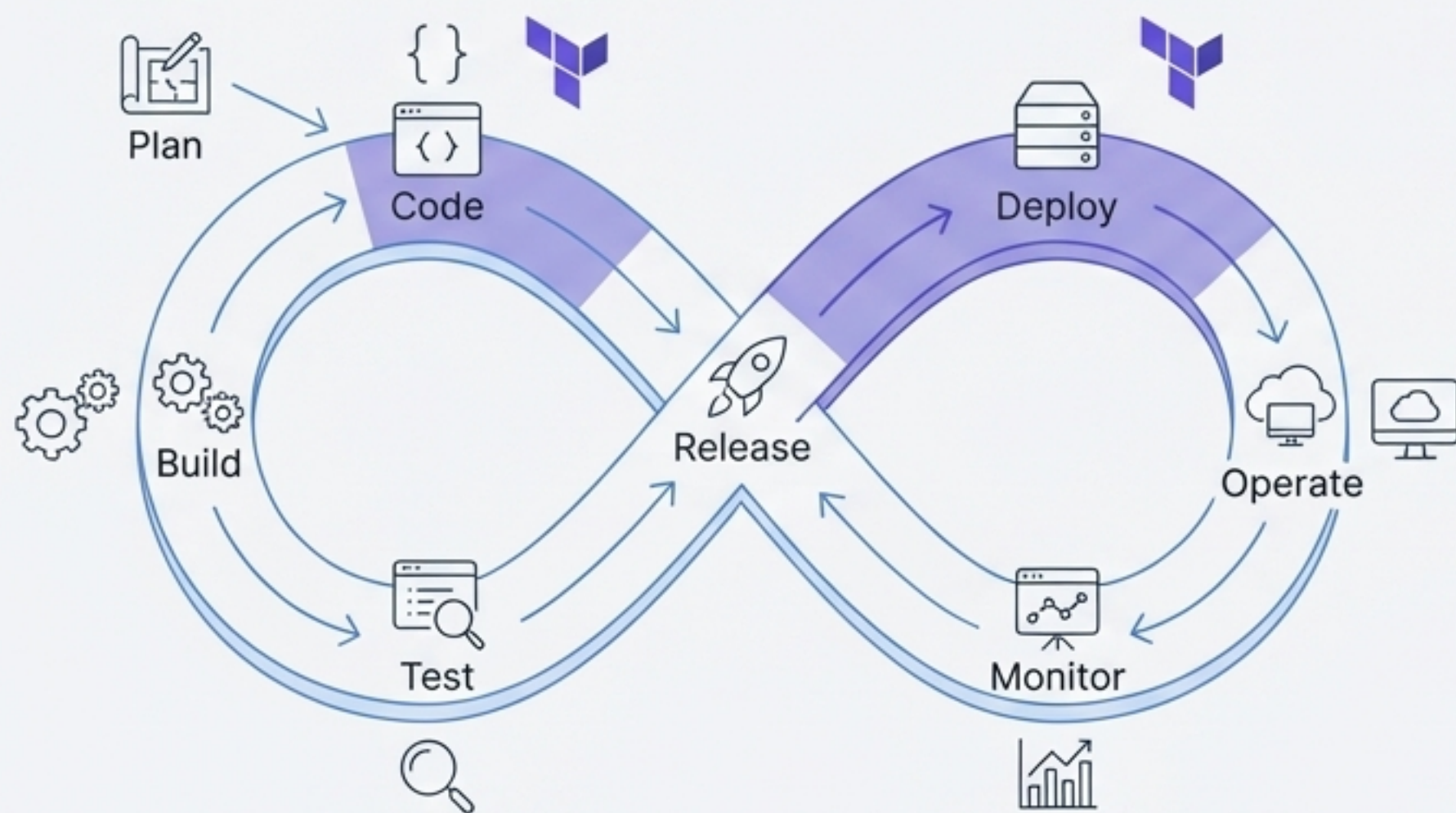
이 아키텍처는 외부에서 SSH 접속이 가능한 웹 서버를 위한 가장 기본적인 구성입니다. Terraform을 사용하면 이 전체 구조를 몇 분 안에 코드로 정의하고 배포할 수 있습니다.

매핑되는 Terraform 리소스

- **VPC:** `resource "aws_vpc" "main"`
- **Public Subnet:** `resource "aws_subnet" "public"`
- **Internet Gateway:**
`resource "aws_internet_gateway" "gw"`
- **Security Group:**
`resource "aws_security_group" "allow_ssh"`
- **EC2 Instance:**
`resource "aws_instance" "web_server"`

더 큰 그림: DevOps 생명주기와 IaC

“데브옵스는 애플리케이션과 서비스를 빠른 속도로 제공할 수 있도록 조직의 역량을 향상시키는 문화 철학, 방식 및 도구의 조합입니다.”



IaC의 역할

코드형 인프라(IaC)는 DevOps의 핵심적인 실천 방식(Practice) 중 하나입니다.

- ↳ **CI/CD (지속적 통합/전달):** 인프라 코드를 애플리케이션 코드와 함께 파이프라인에 통합하여, 테스트 및 배포 환경을 자동으로 프로비저닝합니다.
- ↳ **MSA (마이크로서비스 아키텍처):** 각 마이크로서비스가 필요로 하는 독립적인 인프라(컴퓨팅, 데이터베이스, 큐 등)를 신속하고 일관되게 배포할 수 있습니다.
- ↳ **컨테이너 & Kubernetes:** Terraform을 사용하여 Kubernetes 클러스터(EKS 등) 자체를 프로비저닝하고 관리합니다.

실습 준비: 로컬 개발 환경 구축

클라우드에 적용하기 전, 로컬 한후 로컬 환경에서 안전하고 비용 효율적으로 인프라 코드를 AI 인프라 코드를 테스트하고 개발하는 것이 중요합니다.



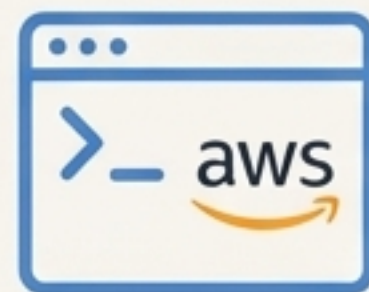
VirtualBox

VirtualBox를하는
호스트 OS(Windows, macOS 등)
위에 독립적인 가상 머신(VM)을
생성하고 실행하는 무료 오픈소스
하이퍼바이저입니다.
“윈도우에 리눅스 추가로 설치!”



Vagrant

VirtualBox와 같은 가상화 인스턴스를
코드로 관리하는 도구입니다.
`Vagrantfile`이라는 설정 파일을
통해 VM 생성, 네트워크 설정,
프로비저닝을 자동화합니다.
“IaC for Hypervisor.”



AWS CLI

명령줄을 통해 AWS 서비스와
상호 작용하는 도구입니다.
Terraform 실행 전후에
Terraform 실행 전후에 AWS 리소스
상태를 확인하거나 간단한 작업을
자동화하는 데 유용합니다.

현대적 인프라 마스터하기: 당신의 여정은 이제 시작입니다.

우리는 전통적인 인프라 관리의 한계에서 시작하여, 클라우드와 IaC라는 새로운 패러다임을 이해했습니다. AWS라는 강력한 플랫폼 위에서 **Terraform**이라는 정교한 설계도를 사용하여, 코드를 통해 인프라를 구축하고 자동화하는 방법을 탐구했습니다. 이것은 단순한 기술이 아니라, 더 빠르고, 안정적이며, 협업에 기반한 새로운 업무 방식입니다.



다음 단계: 직접 구축해보세요

강의 노트 및 추가 정보: https://bit.ly/202506_Terraform

실습 코드 예제:

<https://github.com/Finfra/awsTerraform-course>

<https://github.com/Finfra/vagrant-examples>